

Lecture 10

Introduction to SDP-Based Design of Controllers and Lipschitz Networks

Lecturer: Bin Hu Date: 09/24/2026

So far we have seen that the quadratic constraint approach can be used to formulate semidefinite programming (SDP) conditions for analyzing dynamical systems, iterative optimization algorithms, and Lipschitz neural networks. This lecture introduces how to tailor such analysis conditions for the design of stabilizing controllers and Lipschitz networks.

10.1 SDP-Based Design of Stabilizing Controllers

Consider a simple dynamical system

$$x_{k+1} = Ax_k + Bu_k,$$

where $x_k \in \mathbb{R}^n$ is the state, $u_k \in \mathbb{R}^m$ is the control input, $A \in \mathbb{R}^{n \times n}$, and $B \in \mathbb{R}^{n \times m}$. If A is not Schur stable, i.e., its spectral radius is larger than or equal to 1, then one may ask how to choose u_k as a function of x_k to stabilize the closed-loop dynamics. The mapping from the state measurement to the control input is called the controller. In this lecture, we consider a simple linear state-feedback controller

$$u_k = -Kx_k.$$

Here $K \in \mathbb{R}^{m \times n}$. We assume that (A, B) is *stabilizable*, meaning that some K makes $A - BK$ Schur stable. Such a controller need not exist for an arbitrary pair (A, B) : for example, the scalar system with $A = 2$ and $B = 0$ cannot be stabilized by any control input.

Substituting $u_k = -Kx_k$, the closed-loop dynamics are

$$x_{k+1} = Ax_k - BKx_k = (A - BK)x_k.$$

The closed-loop system is asymptotically stable if and only if there exists a symmetric positive definite matrix $P \succ 0$ such that

$$(A - BK)^\top P (A - BK) - P \prec 0. \quad (10.1)$$

The design of a stabilizing controller involves finding K and P jointly. However, (10.1) is not a linear matrix inequality (LMI) in (P, K) because it contains products between the unknown matrices P and K . Therefore, it cannot be solved directly as an SDP.

To obtain an SDP formulation, we introduce a change of variables. Since $P \succ 0$, pre- and post-multiplying (10.1) by P^{-1} gives

$$P^{-1}(A - BK)^\top P (A - BK)P^{-1} - P^{-1} \prec 0.$$

Define

$$Q = P^{-1}. \quad (10.2)$$

Then the above inequality can be written as

$$((A - BK)Q)^T Q^{-1} ((A - BK)Q) - Q \prec 0. \quad (10.3)$$

By the Schur complement lemma, (10.3) is equivalent to

$$\begin{bmatrix} Q & ((A - BK)Q)^T \\ (A - BK)Q & Q \end{bmatrix} \succ 0. \quad (10.4)$$

The remaining difficulty is that (10.4) still contains the bilinear product KQ . To eliminate this bilinearity, introduce a new decision variable

$$Y = KQ. \quad (10.5)$$

Since

$$(A - BK)Q = AQ - BKQ = AQ - BY, \quad (10.6)$$

condition (10.4) becomes

$$\boxed{\begin{bmatrix} Q & (AQ - BY)^T \\ AQ - BY & Q \end{bmatrix} \succ 0, \quad Q \succ 0.} \quad (10.7)$$

Condition (10.7) is an LMI in the decision variables $Q = Q^T \in \mathbb{R}^{n \times n}$ and $Y \in \mathbb{R}^{m \times n}$. If it is feasible, we recover a Lyapunov matrix and a stabilizing feedback gain as

$$P = Q^{-1}, \quad K = YQ^{-1}.$$

The resulting feedback law

$$u_k = -Kx_k \quad (10.8)$$

then guarantees that $A - BK$ is Schur stable. The change of variables therefore converts the original nonconvex controller-design condition into an equivalent SDP feasibility problem.

10.2 SDP-Based Design of Lipschitz Neural Networks

Next, we use an SDP analysis condition to derive Lipschitz network structures analytically. We first review the classification setting and explain how a Lipschitz bound gives a robustness certificate.

Recall that a map g is L -Lipschitz, for $L \geq 0$, if $\|g(x) - g(z)\| \leq L\|x - z\| \forall x, z$. We use the elementwise ReLU activation $\sigma(t) = \max\{0, t\}$. The arguments below also apply to any elementwise scalar activation that is slope-restricted on $[0, 1]$, meaning

$$0 \leq \frac{\sigma(t_1) - \sigma(t_2)}{t_1 - t_2} \leq 1 \quad \text{for all } t_1 \neq t_2. \quad (10.9)$$

This assumption includes both monotonicity and the 1-Lipschitz property. Consider an image classification task. The goal is to predict the label $y \in \mathcal{Y} := \{1, 2, \dots, q\}$ from the image $x \in \mathbb{R}^d$. A neural network classifier $\mathbf{f} : \mathbb{R}^d \rightarrow \mathbb{R}^q$ generates q real-valued scores, called *logits*, one for each class:

$$\mathbf{f}(x) = \begin{bmatrix} f_1(x) \\ f_2(x) \\ \vdots \\ f_q(x) \end{bmatrix}. \quad (10.10)$$

The logits are unnormalized scores; applying softmax produces a probability vector. The predicted label is $\arg \max_j f_j(x)$, with a fixed rule for breaking ties. For example, in a binary task with cats as class 1 and dogs as class 2, the prediction is “cat” when $f_1(x) > f_2(x)$ and “dog” when $f_2(x) > f_1(x)$. The prediction is correct when it equals the true label y .

There are many ways to parameterize $\mathbf{f}$. Starting from $x_0 = x$, consider h hidden layers, indexed by $k = 0, \dots, h-1$. Two examples are

- Feedforward layers: $x_{k+1} = \sigma(W_k x_k + b_k)$.
- Residual blocks: $x_{k+1} = x_k + \sigma(W_k x_k + b_k)$, with equal input and output dimensions. We will derive a more general residual block below.

As discussed in the class, the use of large weights $\{W_k\}$ can lead to severe sensitivity issues, causing the networks to be fragile to the perturbations in the input space. One way to induce certified robustness is to use the following Lipschitz neural network approach:

- Constructing 1-Lipschitz layers: One can control every layer to be 1-Lipschitz. Then the composition of 1-Lipschitz maps is 1-Lipschitz, and hence the entire neural network is 1-Lipschitz.
- Training and certification: Train the network to increase prediction margins, for example by modifying the loss function [5]. Increasing margins on some examples can affect accuracy on others, so the training hyperparameters must be chosen carefully.

Spectral normalization gives the simplest 1-Lipschitz network structure, but this approach can be conservative. We now design 1-Lipschitz residual blocks using the quadratic constraint approach. Neural network layers can be viewed as special cases of the generalized Lur’e system from the previous lecture:

$$x_{k+1} = A_k x_k + B_k \sigma(C_k x_k + b_k). \quad (10.11)$$

Choosing $A_k = 0$ and $B_k = I$ gives a feedforward layer, while $A_k = I$ gives a residual block. For a residual block with $x_k \in \mathbb{R}^n$, the dimensions are $C_k \in \mathbb{R}^{r \times n}$, $B_k \in \mathbb{R}^{n \times r}$, and $b_k \in \mathbb{R}^r$; the number of nonlinear units r need not equal n .

Under the slope restriction (10.9), the quadratic constraint argument from the previous lectures gives the following sufficient condition [1]: the layer (10.11) is 1-Lipschitz if there exists a positive definite diagonal matrix Λ_k such that

$$\begin{bmatrix} A_k^\top A_k - I & A_k^\top B_k \\ B_k^\top A_k & B_k^\top B_k \end{bmatrix} + \begin{bmatrix} C_k & 0 \\ 0 & I \end{bmatrix}^\top \begin{bmatrix} 0 & \Lambda_k \\ \Lambda_k & -2\Lambda_k \end{bmatrix} \begin{bmatrix} C_k & 0 \\ 0 & I \end{bmatrix} \preceq 0. \quad (10.12)$$

For fixed (A_k, B_k, C_k) , this is an LMI in Λ_k . It is not jointly affine in $(A_k, B_k, C_k, \Lambda_k)$ when the weights are also decision variables. To design a 1-Lipschitz network, we will derive analytical choices of the weights and multipliers that satisfy this condition.

Expanding the quadratic-constraint term gives the equivalent condition

$$\begin{bmatrix} A_k^\top A_k - I & A_k^\top B_k + C_k^\top \Lambda_k \\ B_k^\top A_k + \Lambda_k C_k & B_k^\top B_k - 2\Lambda_k \end{bmatrix} \preceq 0. \quad (10.13)$$

For residual blocks, set $A_k = I$ to obtain

$$\begin{bmatrix} 0 & B_k + C_k^\top \Lambda_k \\ B_k^\top + \Lambda_k C_k & B_k^\top B_k - 2\Lambda_k \end{bmatrix} \preceq 0. \quad (10.14)$$

The zero upper-left block forces the off-diagonal block to vanish. Indeed, if $\begin{bmatrix} 0 & R \\ R^\top & S \end{bmatrix} \preceq 0$, evaluating its quadratic form at $\begin{bmatrix} tRv \\ v \end{bmatrix}$ gives

$$2t\|Rv\|^2 + v^\top S v \leq 0 \quad \text{for every real } t \text{ and every } v.$$

This is possible only if $Rv = 0$ for all v , so $R = 0$; then $S \preceq 0$. Applying this fact to (10.14), we obtain the equivalent requirements

$$B_k = -C_k^\top \Lambda_k, \quad B_k^\top B_k \preceq 2\Lambda_k.$$

Substituting the first relation into the second yields

$$\Lambda_k C_k C_k^\top \Lambda_k \preceq 2\Lambda_k. \quad (10.15)$$

Since $\Lambda_k \succ 0$, congruence by Λ_k^{-1} gives

$$\boxed{\Lambda_k^{-1} \succeq \frac{1}{2} C_k C_k^\top.} \quad (10.16)$$

Consequently, for any C_k and b_k , a diagonal positive definite Λ_k satisfying (10.16) certifies that

$$\boxed{x_{k+1} = x_k - C_k^\top \Lambda_k \sigma(C_k x_k + b_k)} \quad (10.17)$$

is 1-Lipschitz under our activation assumption. This is a sufficient design condition for the layer. In HW2, you will be asked to sort out various choices of Λ_k that satisfy (10.16).

References

- [1] A. Araujo, A. J. Havens, B. Delattre, A. Allauzen, and B. Hu. A unified algebraic perspective on lipschitz neural networks. In *The Eleventh International Conference on Learning Representations*, 2023.
- [2] F. Farnia, J. Zhang, and D. Tse. Generalizable adversarial training via spectral normalization. In *International Conference on Learning Representations*, 2019.
- [3] L. Meunier, B. Delattre, A. Araujo, and A. Allauzen. A dynamical system perspective for lipschitz neural networks. In *International Conference on Machine Learning*, 2022.
- [4] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations*, 2018.
- [5] B. Prach and C. H. Lampert. Almost-orthogonal layers for efficient general-purpose lipschitz networks. In *Computer Vision—ECCV 2022: 17th European Conference*, 2022.